Homework 4 — Machine Learning (Fall 2025)

Yash Dagade

Dec 5, 2025

Agreement. This assignment represents my own work. I have worked with the following people and/or LLMs: Cursor, GPT-5. I would like to thank Alexis Fox, Krish Yadav, Brian Mason, and Max Xiong for high-level discussions about the homework.

Signed, **Yash Dagade**

Kernels — 25 pts (Theory) (Shiyang)

Kernels

In class, we saw that a kernel is defined as

$$K(x_i, x_k) \;=\; \langle \Phi(x_i), \Phi(x_k) \rangle_{\mathcal{H}_K}, \tag{1}$$

for some feature map Φ into a Hilbert space $\mathcal{H}_K$. A kernel can implicitly map data to a higher-dimensional space, which may help algorithms like SVM work better in that space. In this question, you will investigate kernels theoretically and build some intuition about different kernels' effect on SVM.

Recall that K is a (valid) kernel if and only if:

1. K is symmetric:
$$K(x_1, x_2) \;=\; K(x_2, x_1), \tag{2}$$
2. K is positive semidefinite (PSD): for every finite collection $\{x_1, \ldots, x_m\}$ and every $\boldsymbol{c} \in \mathbb{R}^m$,
$$\boldsymbol{c}^\top \mathbf{K} \boldsymbol{c} \;\geq\; 0, \tag{3}$$
where the Gram matrix $\mathbf{K} \in \mathbb{R}^{m \times m}$ is defined by $\mathbf{K}_{ij} = K(x_i, x_j)$.

(a) Sometimes it is very useful to construct a kernel using existing kernels. For this part, assume K, K_1, K_2 are valid kernels. Prove that each of the following constructions defines a valid kernel function:

1. $K'(x_1, x_2) = c\, K(x_1, x_2)$, where $c \geq 0$,
2. $K'(x_1, x_2) = K_1(x_1, x_2)\, K_2(x_1, x_2)$,
3. $K'(x_1, x_2) = f(x_1)\, K(x_1, x_2)\, f(x_2)$, where $f : \mathcal{X} \to \mathbb{R}$.

(b) Using what you proved in part (a), prove that the radial basis function (RBF) kernel is a valid kernel:

$$K(x_1, x_2) \;=\; \exp\left(-\frac{1}{2}\|x_1 - x_2\|^2\right). \tag{4}$$

(c) We now connect kernels with logistic regression. Let $\{(x_i, y_i)\}_{i=1}^n$ be a set of training data, where $x_i \in \mathbb{R}^d$ for all i, and $y_i \in \{-1, 1\}$. Consider ℓ_2-regularized logistic regression with parameter vector θ, where we want to find f_θ that minimizes

$$\sum_{i=1}^{n} \ln\big(1 + \exp(-y_i f_\theta(x_i))\big) \;+\; \lambda \|f_\theta\|_{\mathcal{H}}^2, \tag{5}$$

where $f_\theta(x)$ is of the form $f_\theta(x) = \theta^\top x$, and $\|f_\theta\|_{\mathcal{H}}^2 = \theta^\top \theta$.

1. Let $\mathcal{H}$ be the reproducing kernel Hilbert space (RKHS) corresponding to this ℓ_2-regularized logistic regression. Using the representer theorem, what is the form of the optimal predictive function?

2. Define $g(\zeta) = \ln(1 + \exp(-\zeta))$. It turns out that ℓ_2-regularized logistic regression is closely related to the following primal optimization problem:

$$\min_{\omega,\zeta} \quad \frac{1}{2}\|\omega\|^2 + C\sum_{i=1}^{n} g(\zeta_i) \tag{6}$$

subject to

$$y_i\,(\omega^\top x_i) \;\geq\; \zeta_i, \quad \forall i, \tag{7}$$

where $C > 0$ is a constant.

Derive the dual formulation of this problem (you are not required to solve it). You should simplify the dual so that it depends only on $\boldsymbol{\alpha} = (\alpha_1, \ldots, \alpha_n)$, the labels y_i, the inputs x_i, and C.

Solution

We proceed part by part.

(a) Closure properties of kernels. Let K, K_1, K_2 be valid kernels on a domain $\mathcal{X}$. By definition, for each there exists a feature map into a Hilbert space such that the kernel equals an inner product.

(i) Scaling by $c \geq 0$. Define

$$K'(x_1, x_2) = c\,K(x_1, x_2).$$

Symmetry. Since K is symmetric, for all x_1, x_2,

$$K'(x_1, x_2) = cK(x_1, x_2) = cK(x_2, x_1) = K'(x_2, x_1).$$

Positive semidefinite. Let $\{x_1, \ldots, x_m\}$ be any finite sample from $\mathcal{X}$, and let $\mathbf{K}$ be the Gram matrix of K on these points: $\mathbf{K}_{ij} = K(x_i, x_j)$. The Gram matrix of K' is $\mathbf{K}' = c\,\mathbf{K}$. For any $\boldsymbol{c} \in \mathbb{R}^m$,

$$\boldsymbol{c}^\top \mathbf{K}' \boldsymbol{c} = \boldsymbol{c}^\top (c\mathbf{K})\boldsymbol{c} = c\,\boldsymbol{c}^\top \mathbf{K}\boldsymbol{c} \;\geq\; 0,$$

because $c \geq 0$ and $\mathbf{K}$ is PSD. Hence K' is a valid kernel.

(ii) Product of kernels. Define

$$K'(x_1, x_2) = K_1(x_1, x_2)\,K_2(x_1, x_2).$$

Let $\Phi_1 : \mathcal{X} \to \mathcal{H}_1$ and $\Phi_2 : \mathcal{X} \to \mathcal{H}_2$ be feature maps such that

$$K_1(x_1, x_2) = \langle \Phi_1(x_1), \Phi_1(x_2) \rangle_{\mathcal{H}_1}, \quad K_2(x_1, x_2) = \langle \Phi_2(x_1), \Phi_2(x_2) \rangle_{\mathcal{H}_2}.$$

Consider the tensor-product Hilbert space $\mathcal{H}_1 \otimes \mathcal{H}_2$ and define a new feature map

$$\Psi(x) = \Phi_1(x) \otimes \Phi_2(x).$$

The inner product on tensor products satisfies

$$\langle u_1 \otimes v_1,\; u_2 \otimes v_2 \rangle_{\mathcal{H}_1 \otimes \mathcal{H}_2} = \langle u_1, u_2 \rangle_{\mathcal{H}_1}\,\langle v_1, v_2 \rangle_{\mathcal{H}_2},$$

extended bilinearly.

Then for all $x_1, x_2 \in \mathcal{X}$,

$$\begin{aligned}\langle \Psi(x_1), \Psi(x_2) \rangle_{\mathcal{H}_1 \otimes \mathcal{H}_2} &= \langle \Phi_1(x_1) \otimes \Phi_2(x_1),\, \Phi_1(x_2) \otimes \Phi_2(x_2) \rangle \\ &= \langle \Phi_1(x_1), \Phi_1(x_2) \rangle_{\mathcal{H}_1} \langle \Phi_2(x_1), \Phi_2(x_2) \rangle_{\mathcal{H}_2} \\ &= K_1(x_1, x_2)\, K_2(x_1, x_2) = K'(x_1, x_2).\end{aligned}$$

Thus K' is an inner product in a Hilbert space, hence a valid kernel (symmetric and PSD).

(iii) Multiplication by $f(x_1)f(x_2)$. Let K be a kernel with feature map $\Phi : \mathcal{X} \to \mathcal{H}$:

$$K(x_1, x_2) = \langle \Phi(x_1), \Phi(x_2) \rangle_{\mathcal{H}}.$$

Define

$$K'(x_1, x_2) = f(x_1)\, K(x_1, x_2)\, f(x_2).$$

Define a new feature map

$$\Psi(x) = f(x)\, \Phi(x) \in \mathcal{H}.$$

Then for all x_1, x_2,

$$\begin{aligned}\langle \Psi(x_1), \Psi(x_2) \rangle_{\mathcal{H}} &= \langle f(x_1)\Phi(x_1),\, f(x_2)\Phi(x_2) \rangle \\ &= f(x_1) f(x_2)\, \langle \Phi(x_1), \Phi(x_2) \rangle \\ &= f(x_1) K(x_1, x_2) f(x_2) = K'(x_1, x_2).\end{aligned}$$

Thus K' is also a kernel.

Equivalently, at the Gram-matrix level: for any finite set $\{x_i\}$, let $\mathbf{K}$ be the Gram matrix of K, and let $D = \mathrm{diag}(f(x_1), \ldots, f(x_m))$. Then the Gram matrix of K' is $\mathbf{K}' = D\mathbf{K}D$, and for any $\boldsymbol{c}$,

$$\boldsymbol{c}^\top \mathbf{K}' \boldsymbol{c} = (D\boldsymbol{c})^\top \mathbf{K} (D\boldsymbol{c}) \geq 0,$$

since $\mathbf{K}$ is PSD. So K' is a valid kernel.

(b) The RBF kernel is a valid kernel. We want to show that

$$K(x_1, x_2) = \exp\left(-\frac{1}{2} \|x_1 - x_2\|^2 \right)$$

is a kernel on $\mathbb{R}^d$.

First expand the squared norm:

$$\|x_1 - x_2\|^2 = \|x_1\|^2 + \|x_2\|^2 - 2\, x_1^\top x_2.$$

Hence

$$\begin{aligned}K(x_1, x_2) &= \exp\left(-\frac{1}{2} \left(\|x_1\|^2 + \|x_2\|^2 - 2\, x_1^\top x_2 \right) \right) \\ &= \exp\left(-\frac{1}{2} \|x_1\|^2 \right) \exp\left(-\frac{1}{2} \|x_2\|^2 \right) \exp\left(x_1^\top x_2 \right).\end{aligned}$$

Define:

$$f(x) := \exp\left(-\tfrac{1}{2}\|x\|^2\right), \qquad K_{\text{lin}}(x_1, x_2) := x_1^\top x_2.$$

The linear kernel K_{lin} is a valid kernel (feature map $\Phi(x) = x$).

For any integer $m \geq 0$, consider

$$K_m(x_1, x_2) = \big(K_{\text{lin}}(x_1, x_2)\big)^m = (x_1^\top x_2)^m.$$

Using part (a)(ii), the product of m copies of the linear kernel is a kernel, so each K_m is a kernel.

If $\{K_m\}_{m\geq 0}$ are kernels and $a_m \geq 0$, then

$$\widetilde{K}(x_1, x_2) = \sum_{m=0}^{\infty} a_m K_m(x_1, x_2)$$

is also a kernel, provided the sum converges pointwise. For any finite set of points, the Gram matrix is a sum of PSD matrices with nonnegative weights, hence PSD.

Using the power series expansion,

$$\exp(x_1^\top x_2) = \sum_{m=0}^{\infty} \frac{1}{m!} (x_1^\top x_2)^m = \sum_{m=0}^{\infty} \frac{1}{m!} K_m(x_1, x_2),$$

we see that

$$K_{\exp}(x_1, x_2) := \exp(x_1^\top x_2)$$

is a nonnegative linear combination of kernels, and therefore is itself a kernel.

Finally, by part (a)(iii), multiplying a kernel by $f(x_1)f(x_2)$ yields another kernel. Applying this with $K = K_{\exp}$ and $f(x) = e^{-\|x\|^2/2}$ gives

$$K(x_1, x_2) = f(x_1)\, K_{\exp}(x_1, x_2)\, f(x_2) = \exp\left(-\frac{1}{2}\|x_1 - x_2\|^2\right).$$

Hence the RBF kernel is a valid kernel.

(c) Kernels and logistic regression.

(i) Form of the optimal predictive function via the representer theorem. We consider the problem

$$\min_{f\in\mathcal{H}} \left\{ \sum_{i=1}^{n} \ln\big(1 + \exp(-y_i f(x_i))\big) + \lambda \|f\|_{\mathcal{H}}^2 \right\},$$

where $\mathcal{H}$ is the RKHS associated with some kernel K on $\mathbb{R}^d$ and $f \in \mathcal{H}$ is the decision function.

The representer theorem states that for optimization problems of the form

$$\min_{f\in\mathcal{H}} \Big(\sum_{i=1}^{n} L(y_i, f(x_i)) + \lambda \|f\|_{\mathcal{H}}^2 \Big),$$

where L depends on f only through the values $f(x_i)$, any minimizer f^* admits a representation

$$f^*(\cdot) = \sum_{i=1}^{n} \alpha_i K(x_i, \cdot)$$

for some coefficients $\alpha_1, \ldots, \alpha_n \in \mathbb{R}$.

In our case, $L(y_i, f(x_i)) = \ln(1 + \exp(-y_i f(x_i)))$ depends only on $f(x_i)$, and we have the ℓ_2-norm regularizer $\|f\|_{\mathcal{H}}^2$. Hence, by the representer theorem, the optimal predictive function has the form

$$\boxed{f^*(x) = \sum_{i=1}^{n} \alpha_i \, K(x_i, x)}$$

for some $\alpha_i \in \mathbb{R}$.

In the specific linear case where $K(x, z) = x^\top z$, the RKHS consists of linear functions $f_\theta(x) = \theta^\top x$, and the representation above corresponds to

$$f^*(x) = \sum_{i=1}^{n} \alpha_i x_i^\top x = \Big(\sum_{i=1}^{n} \alpha_i x_i\Big)^\top x,$$

so the optimal parameter vector can be written as

$$\theta^* = \sum_{i=1}^{n} \alpha_i x_i.$$

(ii) Dual formulation of the constrained problem. We are given the primal optimization problem:

$$\begin{aligned} \min_{\omega, \zeta} \quad & \frac{1}{2}\|\omega\|^2 + C \sum_{i=1}^{n} g(\zeta_i) \\ \text{s.t.} \quad & y_i(\omega^\top x_i) \;\geq\; \zeta_i, \quad i = 1, \ldots, n, \end{aligned} \tag{8}$$

where $g(\zeta) = \ln(1 + e^{-\zeta})$.

The constraints can be written as

$$\zeta_i - y_i \, \omega^\top x_i \leq 0, \quad i = 1, \ldots, n.$$

Introduce Lagrange multipliers $\alpha_i \geq 0$ for these constraints.

Lagrangian. The Lagrangian is

$$\begin{aligned} \mathcal{L}(\omega, \zeta, \alpha) &= \frac{1}{2}\|\omega\|^2 + C \sum_{i=1}^{n} g(\zeta_i) + \sum_{i=1}^{n} \alpha_i \big(\zeta_i - y_i \, \omega^\top x_i\big) \\ &= \frac{1}{2}\|\omega\|^2 - \omega^\top \Big(\sum_{i=1}^{n} \alpha_i y_i x_i\Big) + \sum_{i=1}^{n} \big(C g(\zeta_i) + \alpha_i \zeta_i\big). \end{aligned}$$

The dual function is obtained by minimizing $\mathcal{L}$ over the primal variables ω, ζ.

Minimization over ω. The ω-dependent part is

$$\frac{1}{2}\|\omega\|^2 - \omega^\top\Big(\sum_{i=1}^n \alpha_i y_i x_i\Big).$$

This is a strictly convex quadratic in ω with gradient

$$\nabla_\omega \mathcal{L} = \omega - \sum_{i=1}^n \alpha_i y_i x_i.$$

Setting the gradient to zero gives

$$\omega^* = \sum_{i=1}^n \alpha_i y_i x_i.$$

Substituting ω^* back into the quadratic part yields the minimum value

$$\inf_\omega\Big(\frac{1}{2}\|\omega\|^2 - \omega^\top \sum_i \alpha_i y_i x_i\Big) = -\frac{1}{2}\Big\|\sum_{i=1}^n \alpha_i y_i x_i\Big\|^2.$$

So after minimizing over ω, the Lagrangian becomes

$$\inf_\omega \mathcal{L}(\omega, \zeta, \alpha) = -\frac{1}{2}\Big\|\sum_{i=1}^n \alpha_i y_i x_i\Big\|^2 + \sum_{i=1}^n \big(Cg(\zeta_i) + \alpha_i\zeta_i\big).$$

Minimization over ζ. For each i, define

$$h_i(\zeta_i) = Cg(\zeta_i) + \alpha_i\zeta_i.$$

The dual function is

$$\varphi(\alpha) = \inf_{\omega,\zeta} \mathcal{L}(\omega, \zeta, \alpha) = -\frac{1}{2}\Big\|\sum_{i=1}^n \alpha_i y_i x_i\Big\|^2 + \sum_{i=1}^n \inf_{\zeta_i} h_i(\zeta_i).$$

To express $\inf_{\zeta_i} h_i(\zeta_i)$ succinctly, note that this is related to the convex conjugate of g. The convex conjugate g^* of g is defined by

$$g^*(u) = \sup_{\zeta\in\mathbb{R}}\big(u\zeta - g(\zeta)\big).$$

From this definition,

$$\inf_\zeta\big(g(\zeta) + \tfrac{\alpha_i}{C}\zeta\big) = -\sup_\zeta\big(-\tfrac{\alpha_i}{C}\zeta - g(\zeta)\big) = -g^*\big(-\tfrac{\alpha_i}{C}\big)\,.$$

Therefore,

$$\inf_{\zeta_i} h_i(\zeta_i) = C\inf_{\zeta_i}\big(g(\zeta_i) + \tfrac{\alpha_i}{C}\zeta_i\big) = -C\,g^*\big(-\tfrac{\alpha_i}{C}\big)\,.$$

Hence

$$\varphi(\alpha) = -\frac{1}{2}\Big\|\sum_{i=1}^n \alpha_i y_i x_i\Big\|^2 - C\sum_{i=1}^n g^*\big(-\tfrac{\alpha_i}{C}\big)\,.$$

Dual problem (general form). The dual problem is to maximize $\varphi(\alpha)$ over $\alpha_i \geq 0$:

$$\boxed{\begin{aligned} \max_{\alpha \in \mathbb{R}^n} \quad & -\frac{1}{2}\Big\|\sum_{i=1}^n \alpha_i y_i x_i\Big\|^2 - C\sum_{i=1}^n g^*\big(-\tfrac{\alpha_i}{C}\big) \\ \text{s.t.} \quad & \alpha_i \geq 0, \quad i = 1, \dots, n. \end{aligned}}$$

We can make this more explicit. For $g(\zeta) = \ln(1 + e^{-\zeta})$, one can compute (by taking derivatives and solving the first-order optimality condition) that

$$g^*(u) = \begin{cases} (-u)\ln(-u) + (1+u)\ln(1+u), & u \in [-1, 0], \\ +\infty, & \text{otherwise.} \end{cases}$$

Therefore, finiteness of $g^*(-\alpha_i/C)$ requires $-\alpha_i/C \in [-1, 0]$, i.e.

$$0 \leq \alpha_i \leq C.$$

Using this explicit g^*, the dual becomes

$$\begin{aligned} \max_{\boldsymbol{\alpha}} \quad & -\frac{1}{2}\Big\|\sum_{i=1}^n \alpha_i y_i x_i\Big\|^2 \\ & - C\sum_{i=1}^n \Big[\frac{\alpha_i}{C}\ln\Big(\tfrac{\alpha_i}{C}\Big) + \Big(1 - \tfrac{\alpha_i}{C}\Big)\ln\Big(1 - \tfrac{\alpha_i}{C}\Big)\Big] \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad i = 1, \dots, n. \end{aligned}$$

Finally, note that

$$\Big\|\sum_{i=1}^n \alpha_i y_i x_i\Big\|^2 = \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \, x_i^\top x_j,$$

so this dual depends only on (α_i), (y_i), (x_i), and C, as required. In a kernelized version, one would replace $x_i^\top x_j$ by $K(x_i, x_j)$.

Statistical Learning Theory — 40 pts (Theory) (Zack)

Statistical Learning Theory

In lecture, we have viewed the VC dimension on infinite spaces such as $\mathbb{R}^k$, but the definition naturally applies to finite spaces as well.
Let X be a domain, and let $\mathcal{F}$ be a hypothesis space of functions $f : X \to \{-1, 1\}$ on X satisfying

$$f_X \neq g_X, \quad \forall f \neq g \in \mathcal{F}, \tag{9}$$

where f_X denotes the restriction of f onto the domain X. This assumption means that no two functions make all the same predictions on X.
For any subset $P \subset X$, define

$$\mathcal{F}_P = \{f_P \mid f \in \mathcal{F}\}.$$

More simply, $\mathcal{F}_P$ is the set of predictions that functions in $\mathcal{F}$ can make on P. The VC dimension of $(X, \mathcal{F})$ (or just $\mathcal{F}$) is the largest d such that there exists some $P \subset X$ of size d with $|\mathcal{F}_P| = 2^d$ (i.e. P is shattered).
Concretely, consider $X = \{(0, 0), (0, 1), (1, 0), (1, 1)\}$, and let $\mathcal{F}$ be the space of linear decision models on X (with distinct predictions). Then, $|\mathcal{F}_X| = 14$ since no linear decision model can predict the diagonal $\{(0, 0), (1, 1)\}$ as 1 and everything else in X as 0, nor can it predict $\{(1, 0), (0, 1)\}$ as 1 and everything else in X as 0. We then know that the VC dimension of $(X, \mathcal{F})$ is 3 since, if $P = \{(0, 0), (1, 0), (0, 1)\}$, then it can be checked that $|\mathcal{F}_P| = 8$, and $|\mathcal{F}_X| = 14 < 2^4$.
For the rest of this problem, suppose that X is finite of size n and suppose that $\mathcal{F}$ has VC dimension d. Intuitively, the VC dimension of a space $(X, \mathcal{F})$ should be larger when $\mathcal{F}$ contains more models (remember that no two models make identical predictions on X, by assumption). We can make this intuition more rigorous by bounding the size of $\mathcal{F}$ based on its VC dimension.

(a) Show that

$$|\mathcal{F}| \geq 2^d. \tag{10}$$

(b) Let $S_{\mathcal{F}} = \{P \subset X \mid |\mathcal{F}_P| = 2^{|P|}\}$ be the set of subsets of X that are shattered by $\mathcal{F}$ (including the empty set $\emptyset$). Show that

$$|\mathcal{F}| \leq |S_{\mathcal{F}}|. \tag{11}$$

Hint: Use induction on the size of X by removing an element x from X, partitioning $\mathcal{F}$ by how each function classifies x, and applying the induction hypothesis on the resulting partitions. Be very careful with your induction step to not overcount.

Recall the following set identities:

$$|A| + |B| = |A \cap B| + |A \cup B| \tag{12}$$

$$A \subseteq C \text{ and } B \subseteq C \implies |C| - |A \cup B| = |C \setminus (A \cup B)|. \tag{13}$$

(c) Use the previous problem to show that

$$|\mathcal{F}| \leq \sum_{k=0}^{d} \binom{n}{k}. \tag{14}$$

You may not use the (Vapnik and Chervonenkis, Sauer, Shelah) Lemma from class without proof.

Solution

(a) By definition of VC dimension d, there exists a subset $P \subseteq X$ of size $|P| = d$ such that P is shattered by $\mathcal{F}$:

$$|\mathcal{F}_P| = 2^{|P|} = 2^d.$$

Here,

$$\mathcal{F}_P = \{f_P : f \in \mathcal{F}\}$$

is the set of restrictions of functions in $\mathcal{F}$ to P. The map

$$\pi : \mathcal{F} \to \mathcal{F}_P, \quad \pi(f) = f_P,$$

is surjective by definition of $\mathcal{F}_P$. Surjectivity implies

$$|\mathcal{F}| \ \geq \ |\mathcal{F}_P| = 2^d.$$

This is precisely (10).

(b) We prove (11) by induction on $n = |X|$.

Base case: $|X| = 0$.

If $X = \emptyset$, there is exactly one function $f : X \to \{-1, 1\}$ (the empty function). Under the assumption (9), we must have $|\mathcal{F}| \leq 1$. On the other hand, $\mathcal{F}_\emptyset$ contains the unique empty labeling, so $|\mathcal{F}_\emptyset| = 2^0 = 1$, and hence $\emptyset \in S_\mathcal{F}$. Thus $|S_\mathcal{F}| \geq 1$, and $|\mathcal{F}| \leq 1 \leq |S_\mathcal{F}|$, so the inequality holds.

Induction hypothesis. Assume that for every finite domain Y with $|Y| < n$ and every hypothesis class $\mathcal{G}$ on Y satisfying (9), one has

$$|\mathcal{G}| \ \leq \ |S_\mathcal{G}|.$$

Induction step: $|X| = n \geq 1$.

Fix any point $x \in X$ and let $X' = X \setminus \{x\}$.

Partition the class $\mathcal{F}$ according to the label at x:

$$\mathcal{F}^+ = \{f \in \mathcal{F} : f(x) = 1\}, \qquad \mathcal{F}^- = \{f \in \mathcal{F} : f(x) = -1\}.$$

These sets are disjoint and $\mathcal{F} = \mathcal{F}^+ \cup \mathcal{F}^-$, so

$$|\mathcal{F}| = |\mathcal{F}^+| + |\mathcal{F}^-|.$$

Now restrict functions in $\mathcal{F}^+$ and $\mathcal{F}^-$ to X':

$$\mathcal{G}^+ = \{f|_{X'} : f \in \mathcal{F}^+\}, \qquad \mathcal{G}^- = \{f|_{X'} : f \in \mathcal{F}^-\}.$$

These are families of functions defined on the smaller domain X'.

Because we treat $\mathcal{G}^+$ and $\mathcal{G}^-$ as sets of functions on X', if two elements of $\mathcal{G}^+$ were identical on X', they would in fact come from two $f_1, f_2 \in \mathcal{F}^+$ with

$$f_1|_{X'} = f_2|_{X'}, \quad f_1(x) = f_2(x) = 1.$$

Then f_1 and f_2 would agree everywhere on $X = X' \cup \{x\}$, contradicting (9). Thus the restriction map $\mathcal{F}^+ \to \mathcal{G}^+$, $f \mapsto f|_{X'}$, is bijective, and

$$|\mathcal{F}^+| = |\mathcal{G}^+|.$$

The same argument applies to $\mathcal{F}^-$ and $\mathcal{G}^-$, so

$$|\mathcal{F}^-| = |\mathcal{G}^-|.$$

Therefore

$$|\mathcal{F}| = |\mathcal{G}^+| + |\mathcal{G}^-|. \tag{15}$$

Since X' has size $n - 1 < n$, the induction hypothesis yields

$$|\mathcal{G}^+| \le |S_{\mathcal{G}^+}|, \qquad |\mathcal{G}^-| \le |S_{\mathcal{G}^-}|.$$

Combining with (15),

$$|\mathcal{F}| \ \le \ |S_{\mathcal{G}^+}| + |S_{\mathcal{G}^-}|. \tag{16}$$

Next, relate the shattered sets of $\mathcal{G}^+$ and $\mathcal{G}^-$ to those of $\mathcal{F}$. Any $P \subseteq X'$ that is shattered by $\mathcal{G}^+$ satisfies $|\mathcal{G}^+_P| = 2^{|P|}$. But $\mathcal{G}^+_P = \mathcal{F}^+_P$ and $\mathcal{F}^+_P \subseteq \mathcal{F}_P$, so

$$|\mathcal{F}_P| \ge |\mathcal{F}^+_P| = |\mathcal{G}^+_P| = 2^{|P|}.$$

Since $\mathcal{F}_P$ is a set of functions $P \to \{-1, 1\}$, we must also have $|\mathcal{F}_P| \le 2^{|P|}$. Therefore $|\mathcal{F}_P| = 2^{|P|}$ and $P \in S_{\mathcal{F}}$. Hence

$$S_{\mathcal{G}^+} \subseteq S_{\mathcal{F}}.$$

The same argument shows $S_{\mathcal{G}^-} \subseteq S_{\mathcal{F}}$.

Moreover, shattered subsets of $\mathcal{G}^+$ and $\mathcal{G}^-$ live inside X', so they do not contain x. Define

$$S^{\neg x}_{\mathcal{F}} = \{P \in S_{\mathcal{F}} : x \notin P\}, \qquad S^{x}_{\mathcal{F}} = \{P \in S_{\mathcal{F}} : x \in P\}.$$

Then $S^{\neg x}_{\mathcal{F}}, S^{x}_{\mathcal{F}} \subseteq S_{\mathcal{F}}$,

$$S^{\neg x}_{\mathcal{F}} \cap S^{x}_{\mathcal{F}} = \emptyset, \qquad S^{\neg x}_{\mathcal{F}} \cup S^{x}_{\mathcal{F}} = S_{\mathcal{F}},$$

and

$$|S_{\mathcal{F}}| = |S^{\neg x}_{\mathcal{F}}| + |S^{x}_{\mathcal{F}}|. \tag{17}$$

Because $S_{\mathcal{G}^+}, S_{\mathcal{G}^-} \subseteq S^{\neg x}_{\mathcal{F}}$, we have

$$S_{\mathcal{G}^+} \cup S_{\mathcal{G}^-} \subseteq S^{\neg x}_{\mathcal{F}}.$$

Using the identity (12) with $A = S_{\mathcal{G}^+}$ and $B = S_{\mathcal{G}^-}$,

$$|S_{\mathcal{G}^+}| + |S_{\mathcal{G}^-}| = |S_{\mathcal{G}^+} \cap S_{\mathcal{G}^-}| + |S_{\mathcal{G}^+} \cup S_{\mathcal{G}^-}|.$$

The inclusion $S_{\mathcal{G}^+} \cup S_{\mathcal{G}^-} \subseteq S_{\mathcal{F}}^{\neg x}$ implies

$$|S_{\mathcal{G}^+} \cup S_{\mathcal{G}^-}| \leq |S_{\mathcal{F}}^{\neg x}|.$$

Hence

$$|S_{\mathcal{G}^+}| + |S_{\mathcal{G}^-}| \leq |S_{\mathcal{F}}^{\neg x}| + |S_{\mathcal{G}^+} \cap S_{\mathcal{G}^-}|. \tag{18}$$

To control $|S_{\mathcal{G}^+} \cap S_{\mathcal{G}^-}|$, take any $P \in S_{\mathcal{G}^+} \cap S_{\mathcal{G}^-}$. Then $P \subseteq X'$ is shattered by both $\mathcal{G}^+$ and $\mathcal{G}^-$. In particular, for every labeling $\ell : P \to \{-1, 1\}$, there exist:

- $f^+ \in \mathcal{F}^+$ such that $f^+|_P = \ell$ (because P is shattered by $\mathcal{G}^+$),
- $f^- \in \mathcal{F}^-$ such that $f^-|_P = \ell$ (because P is shattered by $\mathcal{G}^-$).

Consider any labeling of $P \cup \{x\}$, which is determined by a labeling ℓ on P and a value $b \in \{-1, 1\}$ at x. If $b = 1$, we may choose f^+ as above: it has $f^+(x) = 1$ and $f^+|_P = \ell$, so it realizes the labeling on $P \cup \{x\}$. If $b = -1$, we choose f^-, which has $f^-(x) = -1$ and $f^-|_P = \ell$, again realizing the labeling.

Thus every labeling of $P \cup \{x\}$ is realized by some $f \in \mathcal{F}$, so $P \cup \{x\}$ is shattered by $\mathcal{F}$. Hence $P \cup \{x\} \in S_{\mathcal{F}}^{x}$.

The map

$$\phi : S_{\mathcal{G}^+} \cap S_{\mathcal{G}^-} \to S_{\mathcal{F}}^{x}, \quad \phi(P) = P \cup \{x\},$$

is injective, since if $P_1 \cup \{x\} = P_2 \cup \{x\}$ with $P_1, P_2 \subseteq X'$, then $P_1 = P_2$. Therefore

$$|S_{\mathcal{G}^+} \cap S_{\mathcal{G}^-}| \leq |S_{\mathcal{F}}^{x}|.$$

Combining this with (18),

$$|S_{\mathcal{G}^+}| + |S_{\mathcal{G}^-}| \leq |S_{\mathcal{F}}^{\neg x}| + |S_{\mathcal{F}}^{x}| = |S_{\mathcal{F}}|$$

by (17). Using (16) we obtain

$$|\mathcal{F}| \leq |S_{\mathcal{G}^+}| + |S_{\mathcal{G}^-}| \leq |S_{\mathcal{F}}|.$$

This completes the induction and proves (11).

(c) Assume $\mathcal{F}$ has VC dimension d. By definition, there exists some subset of X of size d that is shattered, and no subset of size larger than d is shattered. In particular, every shattered subset $P \in S_{\mathcal{F}}$ has size at most d:

$$P \in S_{\mathcal{F}} \implies |P| \leq d.$$

The set $S_{\mathcal{F}}$ is therefore contained in the set of all subsets of X of size at most d, so

$$|S_{\mathcal{F}}| \leq |\{P \subseteq X : |P| \leq d\}| = \sum_{k=0}^{d} \binom{n}{k},$$

since there are $\binom{n}{k}$ subsets of X of size k.

From part (b) we already know $|\mathcal{F}| \leq |S_{\mathcal{F}}|$, hence

$$|\mathcal{F}| \leq |S_{\mathcal{F}}| \leq \sum_{k=0}^{d} \binom{n}{k}.$$

This is exactly (14).

Concentration Inequalities — 20 pts (Theory) (Max)

Concentration Inequalities

(a) Markov's Inequality. Markov's inequality states that for any non-negative random variable X, we have

$$\mathbb{P}(X \geq \epsilon) \leq \frac{\mathbb{E}[X]}{\epsilon} \qquad \text{for any } \epsilon > 0.$$

Prove Markov's inequality.

(b) Chebyshev's Inequality. Chebyshev's inequality states that for any random variable X with mean μ and variance σ^2, we have

$$\mathbb{P}\big(|X - \mu| \geq \epsilon\big) \leq \frac{\sigma^2}{\epsilon^2} \qquad \text{for any } \epsilon > 0.$$

Prove Chebyshev's inequality.

(c) Hoeffding's Inequality (one-sided). A simple version of the Chernoff bound says the following: for a random variable X with mean μ,

$$\mathbb{P}(X - \mu \geq \epsilon) \leq \inf_{\lambda \geq 0} \frac{M_{X-\mu}(\lambda)}{\exp(\lambda\epsilon)},$$

where $M_Z(\lambda) = \mathbb{E}[e^{\lambda Z}]$ is the moment generating function of Z.

Hoeffding's lemma says that for any random variable X taking values in the interval $[a, b]$ (i.e., $\mathbb{P}(a \leq X \leq b) = 1$), the bound

$$\mathbb{E}\big[\exp\big(\lambda(X - \mu)\big)\big] \leq \exp\Big(\frac{\lambda^2(b-a)^2}{8}\Big)$$

holds for every real λ, where $\mu = \mathbb{E}[X]$.

Given $X_1, X_2, \ldots, X_n$ i.i.d. bounded random variables in $[a, b]$ with common mean μ, for any integer $n > 0$ and any real ϵ, the one-sided Hoeffding inequality states:

$$\mathbb{P}\left(\frac{1}{n}\sum_{i=1}^{n} X_i - \mu \geq \epsilon\right) \leq \exp\left(-\frac{2n\epsilon^2}{(b-a)^2}\right).$$

Prove this one-sided version of Hoeffding's inequality. You may use the stated Chernoff bound and Hoeffding's lemma without proof. (Hint: to find the best λ to use, differentiate the exponent with respect to λ and set it to 0.)

Solution

(a) Markov's Inequality Let X be a nonnegative random variable and $\epsilon > 0$. Define the indicator random variable of the event $\{X \geq \epsilon\}$ by

$$\mathbf{1}_{\{X \geq \epsilon\}} = \begin{cases} 1, & X \geq \epsilon, \\ 0, & X < \epsilon. \end{cases}$$

Since $X \geq 0$ and $\epsilon > 0$, the following pointwise inequality holds:

$$X \;\geq\; \epsilon\, \mathbf{1}_{\{X \geq \epsilon\}}.$$

Indeed:

- If $X \geq \epsilon$, then $\epsilon\, \mathbf{1}_{\{X \geq \epsilon\}} = \epsilon \leq X$.
- If $X < \epsilon$, then $\epsilon\, \mathbf{1}_{\{X \geq \epsilon\}} = 0 \leq X$, since $X \geq 0$.

Taking expectations on both sides gives

$$\mathbb{E}[X] \;\geq\; \epsilon\, \mathbb{E}\big[\mathbf{1}_{\{X \geq \epsilon\}}\big] = \epsilon\, \mathbb{P}(X \geq \epsilon).$$

Dividing by $\epsilon > 0$,

$$\mathbb{P}(X \geq \epsilon) \;\leq\; \frac{\mathbb{E}[X]}{\epsilon},$$

which is Markov's inequality.

(b) Chebyshev's Inequality Let X be a random variable with mean $\mu = \mathbb{E}[X]$ and variance $\sigma^2 = \mathrm{Var}(X)$. Consider the nonnegative random variable

$$Y = (X - \mu)^2.$$

Its expectation is

$$\mathbb{E}[Y] = \mathbb{E}\big[(X - \mu)^2\big] = \mathrm{Var}(X) = \sigma^2.$$

For any $\epsilon > 0$, the event $\{|X - \mu| \geq \epsilon\}$ is equivalent to $\{(X - \mu)^2 \geq \epsilon^2\}$, since $|\cdot|$ and squaring are monotone on $[0, \infty)$:

$$\mathbb{P}(|X - \mu| \geq \epsilon) = \mathbb{P}\big((X - \mu)^2 \geq \epsilon^2\big).$$

Applying Markov's inequality to Y with threshold ϵ^2 gives

$$\mathbb{P}\big((X - \mu)^2 \geq \epsilon^2\big) \;\leq\; \frac{\mathbb{E}\big[(X - \mu)^2\big]}{\epsilon^2} = \frac{\sigma^2}{\epsilon^2}.$$

Hence

$$\boxed{\mathbb{P}(|X - \mu| \geq \epsilon) \leq \frac{\sigma^2}{\epsilon^2}}$$

for all $\epsilon > 0$, which is Chebyshev's inequality.

(c) Hoeffding's Inequality (one-sided) Let $X_1, \ldots, X_n$ be i.i.d. random variables taking values in $[a, b]$ with common mean μ. Define

$$\overline{X}_n = \frac{1}{n} \sum_{i=1}^{n} X_i, \qquad S_n = \sum_{i=1}^{n} X_i.$$

Then $\mathbb{E}[S_n] = n\mu$, and the event

$$\overline{X}_n - \mu \geq \epsilon \quad \Longleftrightarrow \quad S_n - n\mu \geq n\epsilon.$$

Thus

$$\mathbb{P}\big(\overline{X}_n - \mu \geq \epsilon\big) = \mathbb{P}\big(S_n - \mathbb{E}[S_n] \geq n\epsilon\big).$$

Apply the Chernoff bound to the random variable S_n with threshold $n\epsilon$: for any $\lambda \geq 0$,

$$\mathbb{P}\big(S_n - \mathbb{E}[S_n] \geq n\epsilon\big) \leq \frac{M_{S_n - \mathbb{E}[S_n]}(\lambda)}{\exp(\lambda n\epsilon)},$$

where

$$M_{S_n - \mathbb{E}[S_n]}(\lambda) = \mathbb{E}\big[\exp\big(\lambda(S_n - \mathbb{E}[S_n])\big)\big].$$

Write

$$S_n - \mathbb{E}[S_n] = \sum_{i=1}^{n} (X_i - \mu).$$

Since the X_i are independent, so are the centered variables $Y_i := X_i - \mu$, and

$$\begin{aligned} M_{S_n - \mathbb{E}[S_n]}(\lambda) &= \mathbb{E}\left[\exp\Big(\lambda \sum_{i=1}^{n} Y_i\Big)\right] \\ &= \prod_{i=1}^{n} \mathbb{E}[\exp(\lambda Y_i)]. \end{aligned}$$

Each $Y_i = X_i - \mu$ still takes values in an interval of length $b-a$, namely $[a-\mu, b-\mu]$. By Hoeffding's lemma, for all real λ,

$$\mathbb{E}\big[\exp(\lambda Y_i)\big] \leq \exp\Big(\frac{\lambda^2 (b-a)^2}{8}\Big).$$

Therefore

$$M_{S_n - \mathbb{E}[S_n]}(\lambda) \leq \left(\exp\Big(\frac{\lambda^2 (b-a)^2}{8}\Big)\right)^n = \exp\Big(\frac{n\lambda^2 (b-a)^2}{8}\Big).$$

Substituting this into the Chernoff bound yields, for every $\lambda \geq 0$,

$$\mathbb{P}\big(\overline{X}_n - \mu \geq \epsilon\big) \leq \exp\Big(\frac{n\lambda^2 (b-a)^2}{8} - \lambda n\epsilon\Big).$$

Define

$$\phi(\lambda) = \frac{n\lambda^2 (b-a)^2}{8} - \lambda n\epsilon.$$

We want to choose $\lambda \geq 0$ to minimize $\phi(\lambda)$, since this gives the tightest bound.

The derivative is

$$\phi'(\lambda) = \frac{n(b-a)^2}{4}\,\lambda - n\epsilon.$$

Setting $\phi'(\lambda) = 0$ gives

$$\frac{n(b-a)^2}{4}\,\lambda - n\epsilon = 0 \quad \Longrightarrow \quad \lambda^* = \frac{4\epsilon}{(b-a)^2}.$$

Because $(b-a)^2 > 0$ and $\epsilon > 0$, we have $\lambda^* > 0$, so this choice is admissible.

Evaluate ϕ at λ^*:

$$\begin{aligned}
\phi(\lambda^*) &= \frac{n(\lambda^*)^2(b-a)^2}{8} - \lambda^* n\epsilon \\
&= \frac{n}{8} \cdot \frac{16\epsilon^2}{(b-a)^4} (b-a)^2 \; - \; n\epsilon \cdot \frac{4\epsilon}{(b-a)^2} \\
&= \frac{n}{8} \cdot \frac{16\epsilon^2}{(b-a)^2} \; - \; \frac{4n\epsilon^2}{(b-a)^2} \\
&= n \cdot \frac{2\epsilon^2}{(b-a)^2} \; - \; n \cdot \frac{4\epsilon^2}{(b-a)^2} \\
&= -\frac{2n\epsilon^2}{(b-a)^2}.
\end{aligned}$$

Since the bound holds for all $\lambda \geq 0$, in particular it holds for $\lambda = \lambda^*$:

$$\mathbb{P}\big(\overline{X}_n - \mu \geq \epsilon\big) \leq \exp\big(\phi(\lambda^*)\big) = \exp\left(-\frac{2n\epsilon^2}{(b-a)^2}\right).$$

Thus the one-sided Hoeffding inequality is established:

$$\boxed{\mathbb{P}\left(\frac{1}{n}\sum_{i=1}^{n} X_i - \mu \geq \epsilon\right) \leq \exp\left(-\frac{2n\epsilon^2}{(b-a)^2}\right).}$$

Note. Question 4 is implemented in Python; the full solution is provided below along with a markdown file that has analysis of our work :)

k_means.py - K-means Clustering Implementation

```
from __future__ import division, print_function
import argparse
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import numpy as np
import os
import random


def init_centroids(num_clusters, image):
    """
    Initialize a `num_clusters` x image_shape[-1] nparray to RGB
    values of randomly chosen pixels of`image`

    Parameters
    ----------
    num_clusters : int
        Number of centroids/clusters
    image : nparray
        (H, W, C) image represented as an nparray

    Returns
    -------
    centroids_init : nparray
        Randomly initialized centroids
    """

    # TODO: Implement init_centroids
    # *** START YOUR CODE ***
    # Get image dimensions
    H, W, C = image.shape

    # Reshape image to (H*W, C) to get all pixels as rows
    pixels = image.reshape(-1, C)

    # Randomly select num_clusters pixels as initial centroids
    random_indices = np.random.choice(pixels.shape[0], num_clusters, replace=False)
    centroids_init = pixels[random_indices].astype(np.float64)
    # *** END YOUR CODE ***

    return centroids_init


def update_centroids(centroids, image, max_iter=30, print_every=10):
    """
    Carry out k-means centroid update step `max_iter` times
```

```
    Parameters
    ----------
    centroids : nparray
        The centroids stored as an nparray
    image : nparray
        (H, W, C) image represented as an nparray
    max_iter : int
        Number of iterations to run
    print_every : int
        Frequency of status update

    Returns
    -------
    new_centroids : nparray
        Updated centroids
    """

    # TODO: Implement update_centroids
    # *** START YOUR CODE ***
    # Get image dimensions and reshape to (H*W, C)
    H, W, C = image.shape
    pixels = image.reshape(-1, C).astype(np.float64)
    num_clusters = centroids.shape[0]

    new_centroids = centroids.copy()

    # Iterate for max_iter iterations
    for iteration in range(max_iter):
        # Initialize array to store cluster assignments
        cluster_assignments = np.zeros(pixels.shape[0], dtype=int)

        # For each pixel, find the closest centroid using Euclidean distance
        for i in range(pixels.shape[0]):
            # Initialize `dist` vector to keep track of distance to every centroid
            dist = np.zeros(num_clusters)

            # Loop over all centroids and store distances in `dist`
            for j in range(num_clusters):
                dist[j] = np.sqrt(np.sum((pixels[i] - new_centroids[j]) ** 2))

            # Find closest centroid and update `new_centroids`
            cluster_assignments[i] = np.argmin(dist)

        # Update `new_centroids` by computing mean of pixels in each cluster
        old_centroids = new_centroids.copy()
        for j in range(num_clusters):
            cluster_pixels = pixels[cluster_assignments == j]
            if len(cluster_pixels) > 0:
                new_centroids[j] = cluster_pixels.mean(axis=0)

        # Print progress
        if (iteration + 1) % print_every == 0:
            print(f'Iteration {iteration + 1}/{max_iter} complete')

        # Check for convergence
        if np.allclose(old_centroids, new_centroids):
```

```
            print(f'Converged at iteration {iteration + 1}')
            break
    # *** END YOUR CODE ***

    return new_centroids


def update_image(image, centroids):
    """
    Update RGB values of pixels in `image` by finding
    the closest among the `centroids`

    Parameters
    ----------
    image : nparray
        (H, W, C) image represented as an nparray
    centroids : int
        The centroids stored as an nparray

    Returns
    -------
    image : nparray
        Updated image
    """

    # TODO: Implement update_image
    # *** START YOUR CODE ***
    # Get image dimensions
    H, W, C = image.shape
    num_clusters = centroids.shape[0]

    # Create output image
    updated_image = np.zeros_like(image, dtype=np.float64)

    # For each pixel, find the closest centroid and replace with centroid value
    for i in range(H):
        for j in range(W):
            pixel = image[i, j].astype(np.float64)

            # Initialize `dist` vector to keep track of distance to every centroid
            dist = np.zeros(num_clusters)

            # Loop over all centroids and store distances in `dist`
            for k in range(num_clusters):
                dist[k] = np.sqrt(np.sum((pixel - centroids[k]) ** 2))

            # Find closest centroid and update pixel value in `image`
            closest_centroid = np.argmin(dist)
            updated_image[i, j] = centroids[closest_centroid]

    # *** END YOUR CODE ***

    return updated_image


def main(args):
```

```
# Setup
max_iter = args.max_iter
print_every = args.print_every
image_path_small = args.small_path
image_path_large = args.large_path
num_clusters = args.num_clusters
figure_idx = 0

# TODO: Load small image
# *** START YOUR CODE ***
image_small = mpimg.imread(image_path_small)
print(f'Loaded small image: {image_small.shape}')
# *** END YOUR CODE ***

# TODO: Initialize centroids
# *** START YOUR CODE ***
centroids = init_centroids(num_clusters, image_small)
print(f'Initialized {num_clusters} centroids')
# *** END YOUR CODE ***

# TODO: Update centroids
# *** START YOUR CODE ***
centroids = update_centroids(centroids, image_small, max_iter, print_every)
print('Centroids updated')
# *** END YOUR CODE ***

# TODO: Load large image
# *** START YOUR CODE ***
image_large = mpimg.imread(image_path_large)
print(f'Loaded large image: {image_large.shape}')
# *** END YOUR CODE ***

# TODO: Update large image with centroids calculated on small image
# *** START YOUR CODE ***
image_compressed = update_image(image_large, centroids)
print('Large image compressed')
# *** END YOUR CODE ***

# TODO: Visualize and save compressed image
# *** START YOUR CODE ***
# Display and save original small image
figure_idx += 1
plt.figure(figure_idx)
plt.imshow(image_small)
plt.title('Original Small Image')
plt.axis('off')
plt.savefig('peppers_small_original.png', bbox_inches='tight', dpi=150)

# Display and save original large image
figure_idx += 1
plt.figure(figure_idx)
plt.imshow(image_large)
plt.title('Original Large Image')
plt.axis('off')
plt.savefig('peppers_large_original.png', bbox_inches='tight', dpi=150)
```

```
# Display and save compressed image
figure_idx += 1
plt.figure(figure_idx)
plt.imshow(image_compressed / 255.0)
plt.title(f'Compressed Image (k={num_clusters})')
plt.axis('off')
plt.savefig('peppers_large_compressed.png', bbox_inches='tight', dpi=150)

print(f'\nImages saved successfully')
# *** END YOUR CODE ***

# Calculate and display compression factor
print("\n" + "=" * 60)
print("COMPRESSION FACTOR CALCULATION")
print("=" * 60)

# Original image representation
H, W, C = image_large.shape
num_pixels = H * W
bytes_per_pixel = 3  # RGB
original_size_bytes = num_pixels * bytes_per_pixel

print("\n### Original Image ###")
print(f"Image dimensions: {H} × {W} pixels")
print(f"Total pixels: {num_pixels:,}")
print(f"Bytes per pixel: {bytes_per_pixel} (RGB channels)")
print(f"Original size: {original_size_bytes:,} bytes")

# Compressed representation
# We have k centroids (colors)
# Each centroid has 3 channels (R, G, B), each 8-bit
# We need to store: (1) the centroids and (2) the cluster assignment for each pixel

import math
centroid_storage = num_clusters * 3  # k centroids, 3 bytes each

# For cluster assignments, we need log2(k) bits per pixel
bits_per_assignment = math.ceil(math.log2(num_clusters))
bytes_per_assignment = bits_per_assignment / 8
assignment_storage = num_pixels * bytes_per_assignment

compressed_size_bytes = centroid_storage + assignment_storage

print("\n### Compressed Image ###")
print(f"Number of clusters (k): {num_clusters}")
print(f"Centroid storage: {num_clusters} centroids × 3 bytes = {centroid_storage} bytes")
print(f"Bits per cluster assignment: {bits_per_assignment} bits (log2({num_clusters}))")
print(f"Assignment storage: {num_pixels:,} pixels × {bytes_per_assignment} bytes = {assignment_storage:
print(f"Total compressed size: {compressed_size_bytes:,.0f} bytes")

# Calculate compression factor
compression_factor = original_size_bytes / compressed_size_bytes

print("\n### Compression Factor ###")
print(f"Compression factor: {original_size_bytes:,} / {compressed_size_bytes:,.0f}")
print(f"                  = {compression_factor:.2f}")
```

```
    print(f"\nThis means the compressed image uses {100/compression_factor:.2f}% of the original size")
    print(f"Space savings: {100 - 100/compression_factor:.2f}%")
    print("=" * 60)

    print('\nCOMPLETE')
    plt.show()


if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--small_path', default='./peppers-small.tiff',
                        help='Path to small image')
    parser.add_argument('--large_path', default='./peppers-large.tiff',
                        help='Path to large image')
    parser.add_argument('--max_iter', type=int, default=150,
                        help='Maximum number of iterations')
    parser.add_argument('--num_clusters', type=int, default=16,
                        help='Number of centroids/clusters')
    parser.add_argument('--print_every', type=int, default=10,
                        help='Iteration print frequency')
    args = parser.parse_args()
    main(args)
```

Question 4: K-means Clustering for Image Compression

Part (a): Implementation

Done in k_means.py

Part (b): Results and Visual Comparison

Original Small Image (128x128)

Figure 1: Original Small Image

Original Large Image (512x512)

Compressed Large Image (16 colors)

Visual Analysis

The compressed image maintains good visual quality with only 16 colors. The peppers' red, yellow, and green tones are well-preserved, and edges remain clearly visible. Some color banding appears in gradient regions, which is expected with quantization to 16 colors.

Part (c): Compression Factor

Original Image

- Size: 512 x 512 = 262,144 pixels
- Storage: 262,144 pixels x 3 bytes (RGB) = **786,432 bytes**

Compressed Image

Storage consists of: 1. **Centroids**: 16 colors x 3 bytes = 48 bytes 2. **Assignments**: 262,144 pixels x 4 bits = 131,072 bytes - Each pixel needs log2(16) = 4 bits to encode its cluster assignment

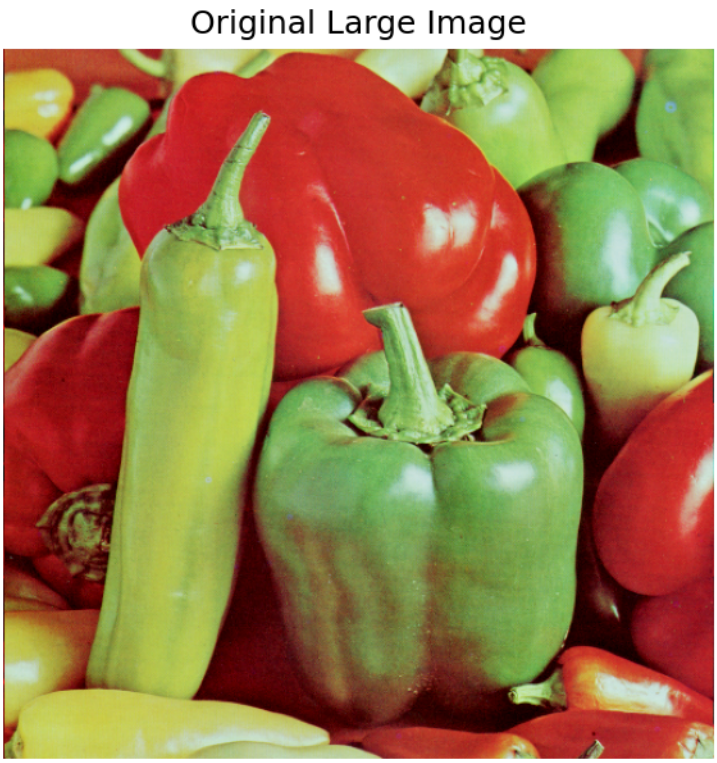

Figure 2: Original Large Image

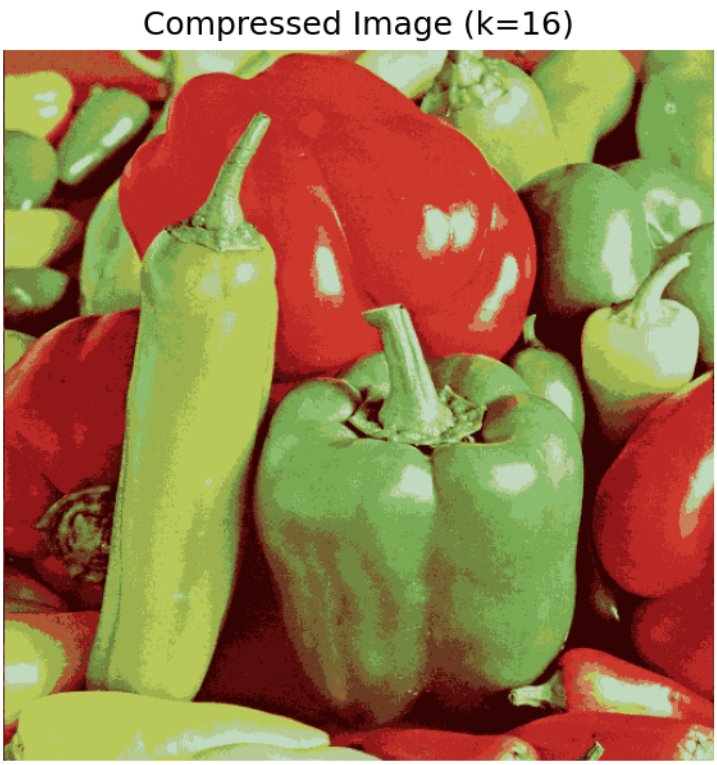

Figure 3: Compressed Large Image

Total compressed size: 48 + 131,072 = **131,120 bytes**

Compression Factor

```
Compression Factor = 786,432 / 131,120 = 6.00
```

The image is compressed by a factor of **6x**, using only 16.67% of the original size with 83.33% space savings.